Lab 1 – SRE Philosophy: Deploy, Break, Understand

Babak Huseynov

June 13, 2026

Overview

QuickTicket was deployed via Docker Compose, the critical path was exercised end-to-end, and each component was killed in turn to observe failure propagation. The gateway was then patched to degrade gracefully when the payments service is unreachable, and a resource-usage study was performed at three load levels: idle, steady load, and load with payment fault injection.

1 Task 1 – Deploy & Break QuickTicket

1.1 All 5 services running (`docker compose ps`)

```
NAME             IMAGE                COMMAND                  SERVICE    STATUS
app-events-1     app-events           "uvicorn main:app ..."   events     Up 16 seconds
app-gateway-1    app-gateway          "uvicorn main:app ..."   gateway    Up 16 seconds
app-payments-1   app-payments         "uvicorn main:app ..."   payments   Up 22 seconds
app-postgres-1   postgres:17-alpine   "docker-entrypoint..."   postgres   Up 22 seconds (
    healthy)
app-redis-1      redis:7-alpine       "docker-entrypoint..."   redis      Up 22 seconds (
    healthy)
```

1.2 Critical path: list → reserve → pay

```
### list events
[
  { "id": 1, "name": "Go Conference 2026", "venue": "Main Hall A",
    "date": "2026-09-15T09:00:00+00:00", "total_tickets": 100,
    "price_cents": 5000, "available": 100 },
  { "id": 4, "name": "Python Workshop", ..., "available": 25 },
  { "id": 2, "name": "SRE Meetup",      ..., "available": 30 },
  { "id": 5, "name": "Kubernetes Deep Dive", ..., "available": 80 },
  { "id": 3, "name": "Cloud Native Summit",  ..., "available": 500 }
]

### reserve 1 ticket for event 1
{
  "reservation_id": "12f8487d-6a95-4aa9-96f6-f1fef3fe9b83",
  "event_id": 1, "quantity": 1,
  "total_cents": 5000, "expires_in_seconds": 300
}

### pay for reservation 12f8487d-6a95-4aa9-96f6-f1fef3fe9b83
{
  "order_id": "12f8487d-6a95-4aa9-96f6-f1fef3fe9b83",
  "event_id": 1, "quantity": 1,
  "total_cents": 5000, "status": "confirmed"
}
```

1.3 Health check (everything green)

```
$ curl -s http://localhost:3080/health | python3 -m json.tool
{
    "status": "healthy",
    "checks": {
        "events": "ok",
        "payments": "ok",
        "circuit_payments": "CLOSED"
    }
}
```

1.4 Dependency map

Derived from reading `app/gateway/main.py`, `app/events/main.py`, and `app/payments/main.py`:

```
client --HTTP--> gateway --HTTP--> events --TCP--> postgres   (orders, events)
                    |                 `---TCP--> redis      (reservation hold + TTL)
                    `-----HTTP-----> payments  (stateless mock charger)
```

Notes from the code:

- `gateway` only fans out to `events` and `payments`; it has no DB or cache of its own.
- `events` owns both data stores: PostgreSQL for canonical events/orders, Redis for the 5-minute reservation hold (`RESERVATION_TTL=300`) and the `event:{id}:held` counter.
- `payments` is stateless and only consulted on the pay leg.
- Pay is a 3-hop choreography: gateway → `payments/charge`, then gateway → `events/reservations/{id}/cor` which atomically inserts the order in Postgres and clears the Redis hold.

1.5 Systematic failure exploration

Each component was stopped via `docker compose stop <svc>`, the four endpoints were probed, then the component was restarted. The pay column uses a dummy reservation ID (no Redis lookup needed to reach the failure mode).

Killed	List events	Reserve	Pay	Healt
`payments`	200 OK	200 OK	502 “Payment service unavailable”	503 `d`
`events`	502 “Events service unavailable”	502	500 “confirmation failed”	503 `e`
`redis`	200 OK	504 “Events service timeout”	500 “confirmation failed”	503 `e`
`postgres`	502 “Events service unavailable”	500 (internal)	500 “confirmation failed”	503 `e`

Observations:

- **Blast radius is asymmetric.** Killing `payments` only affects the pay endpoint (reads and reservations stay healthy). Killing `postgres` or `events` takes almost everything down.
- **Redis is a soft dependency for reads**, hard for writes. `/events` keeps returning data because availability is computed from Postgres counts; but `reserve` times out at 504 because it tries to `SETEX` into Redis with the default 1 s socket timeout (`REDIS_TIMEOUT_MS=1000`).

- **The health endpoint reflects every failure correctly**: it returns 503 with the specific dependency marked `down`/`degraded`, which would make this a usable readiness probe in production.

1.6 Load generator output

Baseline – 5 req/s for 30 s with the system healthy:

```
QuickTicket Load Generator
Target: http://localhost:3080 | RPS: 5 | Duration: 30s
---
[10s] requests=39 success=39 fail=0 error_rate=0%
[20s] requests=79 success=79 fail=0 error_rate=0%
---
Done. total=117 success=117 fail=0 error_rate=0%
```

Same load with `docker compose stop payments` fired ~8 s into the run:

```
[10s] requests=38 success=37 fail=1   error_rate=2.6%
[20s] requests=78 success=72 fail=6   error_rate=7.6%
---
Done. total=117 success=101 fail=16  error_rate=13.6%
```

The error rate climbs from 0% to 13.6% as the 10% "full purchase flow" traffic and 20% of reserves that hit a dead payments service start failing. The 70% read traffic remains unaffected, which matches the dependency map.

2 Task 2 – Graceful Degradation

The pay handler in `app/gateway/main.py` previously fell through to the generic `Exception` arm and returned a bare 502. It now catches `httpx.ConnectError` specifically and returns a structured 503 with the reservation ID so the client can retry.

2.1 Diff (`git diff app/gateway/main.py`)

```
diff --git a/app/gateway/main.py b/app/gateway/main.py
@@ -336,6 +336,16 @@ async def pay_reservation(reservation_id: str):
         raise HTTPException(504, "Payment service timeout")
     except httpx.HTTPStatusError as e:
         raise HTTPException(e.response.status_code, "Payment failed")
+    except httpx.ConnectError as e:
+        log.warning(f"payments unreachable, degrading gracefully: {e}")
+        return JSONResponse(
+            status_code=503,
+            content={
+                "error": "payments_unavailable",
+                "message": "Payment service is temporarily down. Your "
+                           "reservation is held -- try again in a few minutes.",
+                "reservation_id": reservation_id,
+            },
+        )
     except Exception as e:
         log.error(f"payment error: {e}")
         raise HTTPException(502, "Payment service unavailable")
```

2.2 Verification with payments stopped

```
$ docker compose stop payments
 Container app-payments-1 Stopped

$ curl -s -w "\nHTTP=%{http_code}\n" -X POST \
    http://localhost:3080/events/1/reserve \
    -H 'Content-Type: application/json' -d '{"quantity":1}'
{"reservation_id":"b5f0e9bf-e132-4989-945f-60a7d0cb01b4","event_id":1,
 "quantity":1,"total_cents":5000,"expires_in_seconds":300}
HTTP=200

$ curl -s -w "\nHTTP=%{http_code}\n" -X POST \
    "http://localhost:3080/reserve/b5f0e9bf-e132-4989-945f-60a7d0cb01b4/pay"
{"error":"payments_unavailable",
 "message":"Payment service is temporarily down. Your reservation is held
            -- try again in a few minutes.",
 "reservation_id":"b5f0e9bf-e132-4989-945f-60a7d0cb01b4"}
HTTP=503
```

Reserve still returns 200 (the call only touches events), pay now returns a clean, machine-parseable 503 with the reservation ID echoed back, so a UI can offer "retry payment" without losing the user's hold.

3 Task 3 – GitHub Community Engagement

GitHub Community

Starring a repository is the cheapest way to bookmark useful projects and, in aggregate, signal to maintainers that their work is worth investing time in. For a tool like `simple-container-com/api`, every star nudges visibility in GitHub's search and trending pages, which is how small open-source projects find new contributors. Following developers turns GitHub into a passive feed of what people in your circle are actually building: PRs, releases, and forks show up in your activity stream, so when a teammate or future collaborator publishes something relevant – a new branching strategy, a library, or a postmortem – you see it without having to ask. In a team setting that signal speeds up onboarding to a stranger's codebase, and in a professional setting it builds a network of people whose technical taste you can rely on.

4 Bonus Task – Resource Usage Under Load

4.1 Stats tables (`docker stats -no-stream`)

Idle – all 5 containers running, no traffic:

```
NAME             CPU %    MEM USAGE / LIMIT     NET I/O           PIDS
app-gateway-1    0.27%    39.55MiB / 3.813GiB   405kB / 392kB     2
app-events-1     0.25%    43.23MiB / 3.813GiB   318kB / 426kB     2
app-postgres-1   0.00%    23.70MiB / 3.813GiB   174kB / 198kB     8
app-redis-1      0.60%    3.586MiB / 3.813GiB   51.9kB / 20.9kB   6
app-payments-1   0.31%    33.44MiB / 3.813GiB   1.36kB / 715B     2
```

Under load – loadgen at 10 req/s, snapshot mid-run, payments healthy:

```
NAME             CPU %    MEM USAGE / LIMIT     NET I/O           PIDS
app-gateway-1    2.95%    39.60MiB / 3.813GiB   578kB / 559kB     2
app-events-1     1.52%    43.29MiB / 3.813GiB   464kB / 625kB     2
app-postgres-1   1.76%    23.77MiB / 3.813GiB   256kB / 297kB     8
app-redis-1      1.51%    3.586MiB / 3.813GiB   67.1kB / 27.4kB   6
app-payments-1   0.38%    33.52MiB / 3.813GiB   4.43kB / 2.83kB   2
```

Chaos – loadgen at 10 req/s with `PAYMENT_FAILURE_RATE=0.3`, `PAYMENT_LATENCY_MS=500`:

```
NAME             CPU %    MEM USAGE / LIMIT     NET I/O           PIDS
app-gateway-1    1.75%    39.71MiB / 3.813GiB   849kB / 823kB     2
app-events-1     0.70%    43.30MiB / 3.813GiB   696kB / 935kB     2
app-postgres-1   0.24%    23.74MiB / 3.813GiB   386kB / 452kB     8
app-redis-1      0.63%    3.594MiB / 3.813GiB   91.8kB / 37.6kB   6
app-payments-1   0.16%    34.74MiB / 3.813GiB   3.25kB / 2.26kB   2
```

4.2 Analysis

Most memory: `events` is the heaviest service at 43 MiB, both at rest and under load. It is the only Python process that imports both `psycopg2` (connection pool) and the `redis` client, plus FastAPI; `gateway` (httpx + FastAPI) sits at ~39 MiB and `payments` (FastAPI only) at ~33 MiB. Memory is dominated by the loaded Python interpreter and these libraries – it barely moves between idle and 10 RPS, which is consistent with the application doing essentially no allocation per request.

Most CPU under load: `gateway` (2.95%). That matches the architecture: every request enters here, gets routed, and on the pay path the gateway makes two outbound HTTP calls and assembles a response. It does ~2× the work of `events` (which only handles inbound requests) and an order of magnitude more than `payments` (which most requests do not touch – only the 10% full-purchase traffic).

Fault injection in payments shifts cost upstream. Under chaos, payments' own CPU drops (0.16%) because half a second of each request is spent in `time.sleep`, which is idle in Linux scheduling terms. But the *gateway* sees its network I/O nearly double (578 kB→849 kB outbound) while serving the *same* request rate – because slow payments hold gateway connections open longer, so retries and concurrent open httpx sockets pile up. This is the textbook "slow dependency" pattern: the dependency looks idle, the caller looks busier, and a real production gateway would eventually hit its file-descriptor or connection-pool limit and start dropping unrelated traffic.